# Timeline Scheduler for FreeRTOS

2025/26

Operating systems for embedded systems

Lorenzo Giandomenico

Alessandro Minutoli

Mattia Giosa

Federico Capitani

Ludovico Rigotti

# 1 Project goal

The primary objective of this project is to design and implement a **deterministic, timeline-based scheduler** intended to extend the default priority-based scheduler in FreeRTOS. By adopting a **Timeline scheduler**, the system enforces task execution based on a strict global timeline divided into Major Frames and Sub-frames. The scheduler orchestrates two distinct task categories: **Hard Real-Time (HRT)** and **Soft Real-Time (SRT)**.

- **HRT Tasks:** These are strictly time-critical. They are activated at a precise start time relative to the sub-frame and must complete within a defined "Hard Window" (deadline). If an HRT task exceeds its allocated slot, it is forcibly terminated to preserve the timeline integrity.
- **SRT Tasks:** These operate on a **best-effort basis**, utilizing the idle time remaining after HRT execution. They follow a fixed, compile-time defined sequence. Unlike HRT tasks, SRT tasks offer no guarantee of completion within a specific frame; they are simply preempted when an HRT slot begins.

To ensure predictability, the task model relies on a "run-to-completion" paradigm without internal self-rescheduling logic. Inter-task communication is restricted to **polling mechanisms** to avoid blocking states that could violate deterministic timing. Furthermore, to guarantee repeatability, the entire system state is reset at the end of every Major Frame, reinitializing all tasks for the next cycle.

From an implementation standpoint, the system relies on a custom configuration structure (struct) that defines all static parameters (name, function, type, timing). Crucially, this deterministic behavior was achieved by **directly modifying the core scheduling logic within tasks.c**. This native kernel-level integration ensures that the timeline dictates context switching with maximum precision, bypassing the standard FreeRTOS ready-list selection algorithm.

# 2 Timeline Implementation

To implement the Timeline Scheduler, the system architecture required a fundamental modification of the FreeRTOS kernel, specifically targeting the core scheduling routine `vTaskSwitchContext` within `tasks.c`. The first critical adaptation ensures the scheduler evaluates the timeline at every system tick. By modifying `xTaskIncrementTick` (called by the hardware xPortSysTickHandler ISR) to force `xSwitchRequired = pdTRUE`, we guarantee that `vTaskSwitchContext` is invoked on every tick interrupt. This transforms the default event-driven behavior into a time-triggered paradigm.Within `vTaskSwitchContext`, the standard priority-based selection algorithm (macro `taskSELECT_HIGHEST_PRIORITY_TASK`) has been replaced by a deterministic timeline logic, conditionally compiled using the `configUSE_TIMELINE` flag. The logic proceeds as follows:

- **Cyclic Time Calculation:** The scheduler calculates the relative time within the Major Frame (`xRelTick`) using modulo arithmetic: `xRelTick = xTickCount % TIMELINE_PERIOD`. This creates a recurring time window structure.
- **Task Selection Loop:** The system iterates through the `pxTimelineTasks` list (populated during `xTaskCreate`) to identify the candidate for execution.

**2.2 Hard Real-Time (HRT) Enforcement** For every HRT task, the scheduler checks if `xRelTick` falls within its assigned window (`Start <= xRelTick <= End`).

- **Active Window:** If the current time is within an HRT window, the flag `xHardWindowActive` is set to `pdTRUE`. This effectively locks out Soft Real-Time (SRT) tasks, enforcing strict isolation.
- **Early Completion & Slack Stealing:** A crucial feature of this implementation is the dynamic reclaiming of idle time. The scheduler checks if the HRT task assigned to the current window is already in the `xSuspendedTaskList` (indicating it completed its function early). If so, the window is treated as 'inactive' despite the time check, and `xHardWindowActive` remains `pdFALSE`. This allows SRT tasks to execute immediately, utilizing the 'slack time' created by the early completion of the HRT task.

## 2.4 Periodic Repetition

At the end of each Major Frame, the state of every task is reset to its initial value. This is achieved via the `xTaskReset` function. First, the Stack Pointer is manually reset to the top of the allocated stack memory using the following formulas:

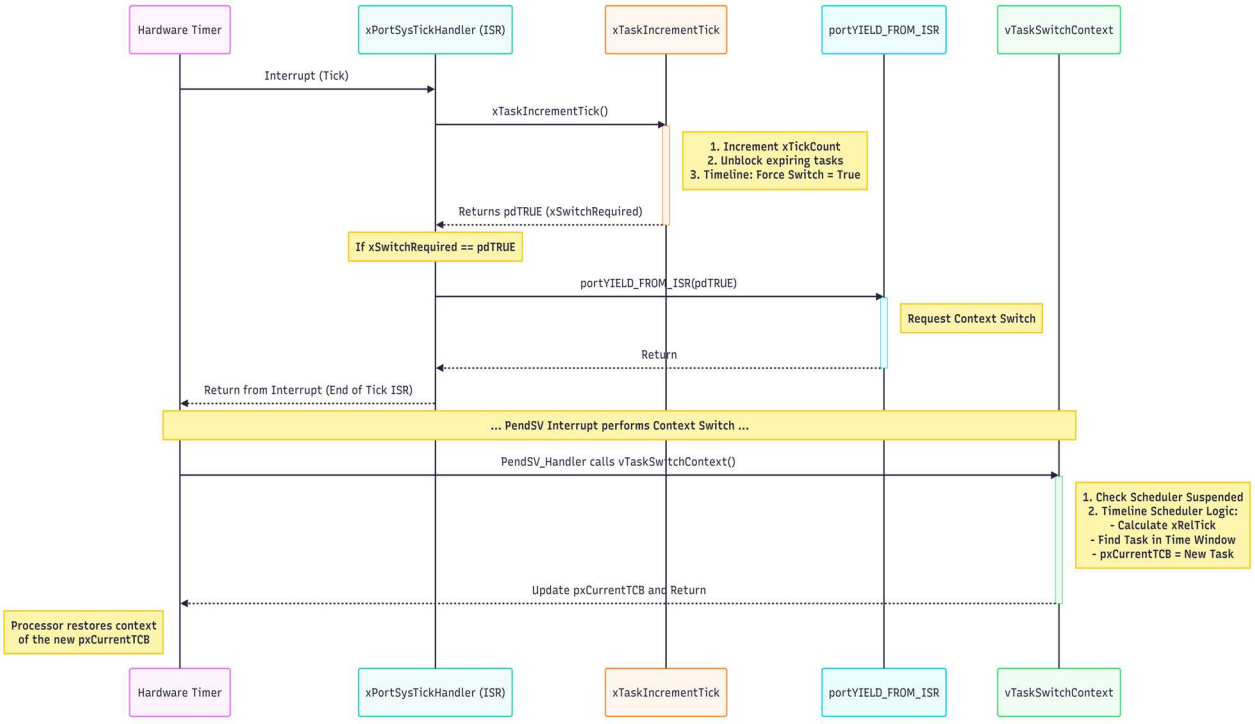
The pointer is moved to the limit of the allocated memory block:

$$SP_{initial} = BaseAddress + (StackDepth - 1) \times SizeOf(StackType)$$

The pointer is then aligned to satisfy the hardware's architecture requirements:

$$SP_{aligned} = SP_{initial} \wedge (portBYTEALIGNMENTMASK)$$

Next, the Stack Frame is re-initialized by setting the Program Counter to the top of the stack and loading the correct `pvParameters` into the CPU registers. Finally, each task is removed from any state list (e.g., Blocked, Suspended) and placed directly into the Ready List. `xTaskReset` is called for every task at the conclusion of `xTaskIncrementTick`.



**2.5 Configuration Interface** All scheduling parameters are defined within a dedicated OS data structure: `TimelineTaskConfig_t`. This structure includes the following fields:

- `pcName`: Task name;
- `pxTaskCode`: Function to execute;
- `pvParameters`: Parameters;
- `uxStackDepth`: Stack Size;
- `xTimeStart` and `xTimeEnd`: Start and End Tick;
- `xTaskType`: Task's type, HARD or SOFT;
- `xSemHandler`: Semaphore Handler.

Subsequently, the system call `vConfigureScheduler` was implemented to instantiate tasks using the `pxScheduleTable` provided by the user. This parameter is a pointer to an array of `TimelineTaskConfig_t` structures. The array must be terminated by a sentinel entry (a row initialized with NULL values). Since the scheduler iterates through this array dynamically during initialization, this sentinel is required to reliably detect the end of the configuration list. The tasks are kept in the Ready List, the `vTaskSwitchContext` function will then decide if they should run immediately. `vConfigureScheduler` also checks if each task is schedulable (End > Start). If a task is invalid, an error is printed to the output, but the initialization loop continues to test system robustness. Additionally, if `configDEBUG_TIMELINE` is set to 1, a debug routine verifies schedule feasibility by calculating the Least Common Multiple (LCM) and Greatest Common Divisor (GCD) of the timing parameters.

Both `TimelineTaskConfig_t` and `vConfigureScheduler` are defined in timeline.h.

## 2.6 Debug flag

Given the macro defined in `FreeRTOSConfig.h`: `#define configDEBUG_TIMELINE 0.` This option enables additional kernel-level debug features when set to 1. It is intended for testing and validating extra timeline-related functionality during development. One of the debug features we implemented is the internal computation of CPU usage. Another implementation of this procedure is presented in **[3.1]**.

## 2.7 Priority based

If the macro defined in `FreeRTOSConfig.h` is set to: `#define configUSE_TIMELINE 0`. The scheduler falls back to the standard FreeRTOS scheduling policy (priority-based). This behavior is implemented using #if / #else conditional compilation: when `configUSE_TIMELINE` is disabled, the timeline-specific code is excluded and the original FreeRTOS kernel code path is compiled (copied verbatim from the upstream kernel to preserve the default behavior).

## 2.8 Handling semaphores

If the macro defined in `FreeRTOSConfig.h` is set to: `#define configUSE_TL_FORCE RESETSEMAPHORE 1`. The scheduler will automatically release the semaphores registered for handling at the end of each major cycle. This feature is useful when tasks must have the resources available at every iteration (e.g., at the beginning of each new major cycle). In that case, the scheduler forces the release of the registered semaphores at the end of the major cycle.

You can choose to manage **all** semaphores or only a **subset**. The scheduler maintains a static list of semaphores: `staticSemaphoreHandle_txListSemaphoreHandler[MAX_NUM_SEM]`

At the end of the major cycle, it releases all non-NULL semaphores in the list. To register semaphores, pass them as a parameter in the task configuration structure provided to `vConfigureScheduler`. The scheduler stores the semaphore handle into `xListSemaphoreHandler`.

In summary, when `configUSE_TL_FORCERESETSEMAPHORE` is enabled, the scheduler can force the release of selected semaphores at the end of each major cycle, ensuring resources are available at the next iteration.

# 3 Test Cases

This document outlines the automated test suite developed to validate the correctness, robustness, and timing precision of the custom FreeRTOS Timeline Scheduler. The test suite comprises seven distinct scenarios, ranging from basic timing verification to stress testing and error handling enforcement. Each test is designed to target specific requirements of Time-Triggered Architectures (TTA), such as determinism, preemption, and temporal isolation.

**Test 1 Overlapping HRT Tasks (Stress Test)**

**Objective:** The primary objective of this test is to evaluate the scheduler's robustness when resolving scheduling conflicts. It aims to verify how the kernel handles configuration errors or aggressive scheduling where two Hard Real-Time (HRT) tasks are assigned overlapping execution windows.

**Description:** In this scenario, two HRT tasks are scheduled with conflicting timelines: *Task A* is assigned the window [0–100 ticks], and *Task B* is assigned [50–150 ticks]. Under normal circumstances, a strictly deterministic schedule should not have overlaps. However, this "stress test" forces the condition to ensure the system remains stable. The test validates that the scheduler does not crash or deadlock when valid windows overlap and ensures that both tasks are eventually executed within the major frame, maintaining system responsiveness despite the configuration stress.

**Test 2: Minimal Time Gaps (Boundary Condition)**

**Objective:** This test validates the scheduler's ability to handle contiguous Hard Real-Time (HRT) windows without inserting idle gaps and enforces strict preemption of Soft Real-Time (SRT) tasks during critical slots.

**Description:** The schedule configures a CPU-bound background SRT task (*Task C*) overlapping with two distinct HRT sequences (*Tasks A-B* and *Tasks D-E*). The test verifies that the scheduler performs seamless context switches between back-to-back HRT tasks (at tick 100) and strictly suppresses the SRT workload whenever a hard window is active. *Task C* is expected to run exclusively during the idle intervals left by the critical tasks. The test passes if all HRT tasks execute within their exact boundaries without drift, and the SRT task execution counter increments only during the calculated idle windows (e.g., [200–450]).

**Test 3: SRT Preemption (Priority Logic)**

**Objective:** This test validates the scheduling hierarchy, ensuring that Hard Real-Time (HRT) tasks strictly preempt Soft Real-Time (SRT) tasks. It confirms that the system respects the "Hard Window" logic, where critical tasks take precedence over background workloads.

**Description:** The system is configured with a long-running SRT task scheduled for the entire window [0–200 ticks]. At tick 100, an HRT task is scheduled to begin [100–150 ticks]. The expected behavior is that the SRT task starts immediately at tick 0. However, exactly at tick 100, the scheduler must interrupt (preempt) the SRT task to allow the HRT task to run. Once the HRT task completes its work, control may return to the SRT task. The test passes only if the HRT task successfully interrupts the soft task and starts execution within its designated critical window.

**Test 4: Timing Consistency (Jitter Measurement)**

**Objective:** In Real-Time Systems, predictability is as important as speed. This test measures the "release jitter" of the scheduler—the variance between the scheduled start time and the actual execution start time.

**Description:** A single HRT task is scheduled to start at a specific offset (tick 50) and run until tick 150. The test logic records the precise tick count when the task actually begins executing code. It calculates the variance (jitter) between the requested start time (50) and the actual start time. The test enforces a strict pass/fail criterion: the variance must be less than or equal to 5 ticks. This proves that the scheduler is deterministic and can trigger tasks reliably.

**Test 5: Multiple Sequential HRT (Context Switch Stress)**

**Objective:** This test evaluates the stability of the scheduler over a sequence of context switches. It aims to ensure that the internal state machine remains consistent when transitioning through a chain of multiple distinct tasks.

**Description:** The schedule is populated with a "train" of five sequential HRT tasks (S0 through S4), arranged back-to-back (e.g., 0–100, 100–200, 200–300, etc.). Each task performs a workload and increments a shared counter. The test verifies that all five tasks execute in the correct order and that the final state reflects the completion of the entire sequence. This validates that the scheduler can handle a busy timeline without losing track of the task index or corrupting the stack during frequent context switching.

**Test 6: Deadline Miss Violation (Temporal Isolation)**

**Objective:** This is a negative test designed to verify the safety enforcement mechanisms of the kernel. The objective is to prove that the scheduler enforces "Temporal Isolation"—preventing a task from exceeding its allocated time slot and starving other processes.

**Description:** A "rogue" task is created with a short window [0–20 ticks] but is programmed to perform a workload that takes 40 ticks to complete. Under a standard priority scheduler, this task would hog the CPU and delay subsequent tasks. However, in this Timeline Scheduler, the kernel is expected to detect that the task is still running at tick 20 and forcibly terminate or deschedule it.

- **Success Condition:** The task is killed before it finishes its work. The test reports "PASSED" if the task's completion flag is *never* set.
- **Failure Condition:** If the task manages to finish its 40-tick workload, it means the scheduler failed to enforce the deadline.

**Test 7: Semaphore Handler - Semaphore Persistence and Resource Locking**

**Objective:** This test validates the integration of standard FreeRTOS synchronization primitives within the custom Timeline Scheduler. The primary objective is to verify that resource states (such as a binary semaphore being "taken") persist correctly across different timeline windows and context

switches. It ensures that the scheduler does not inadvertently reset synchronization objects when switching between tasks.

**Description:** The scenario involves two distinct tasks sharing a single binary semaphore.

1. **Acquisition Phase:** The first task (Soft Real-Time) runs in the initial window [0–400 ticks]. It successfully acquires the semaphore but intentionally does *not* release it before suspending itself.
2. **Verification Phase:** The second task (Hard Real-Time) runs in a subsequent window [500–900 ticks]. It attempts to acquire the same semaphore with a short timeout.
   - **Success Condition:** The second task fails to acquire the semaphore (reporting "STILL TAKEN"). This confirms that the kernel correctly tracked the semaphore's state as "owned" by the first task throughout the timeline transition.
   - **Failure Condition:** If the second task successfully acquires the semaphore, it indicates that the synchronization state was lost or corrupted between windows.

**Test 8: Polling simulation**

**Objective**: Given our fixed-priority scheduling architecture, this resource does not necessitate mutual exclusion mechanisms like locks or semaphores; the scheduler ensures that no process can access the resource while it is already in use. Consequently, there is no risk of overlap or preemption.
While the system architecture prevents concurrent access by design, any logic-level error handling remains the responsibility of the end-user. Ultimately, we successfully simulated a global variable accessed by various tasks in distinct, isolated time slots, while reserving traditional synchronization primitives exclusively for critical resources

**Description**: Two tasks access a global counter. Task A increments the counter and checks if Task B has also performed its increment. If the counter reaches 2, the test is marked as PASSED and the counter is reset to its original value to allow for testing in the subsequent major cycle.

- **Success Condition**: The tGlobalCounter is equal to 2.
- **Failure Condition**: The tGlobalCounter is not equal to 2.

# 4 Configuration and Automation Tools

To streamline the deployment of the deterministic timeline, the system includes a dedicated configuration suite. This tooling abstracts the scheduling complexity, allowing for a separation between the high-level design and the low-level kernel implementation. By using automated generation, the system ensures that the static parameters required by the struct are validated before compilation.

## 4.2 System Configuration (system_config.json)

The scheduling parameters are defined within a structured JSON metadata file. This file serves as the single source of truth for Major Frame architecture.

- **Global Parameters**: Defines the `system_name`, the `timeline_period` (Major Frame duration in ticks), and the `tick_rate_hz` to align the scheduler with the hardware timer.
- **Task Definitions**: Each entry in the tasks array populates the static configuration structure with the following fields:
  - `name`: A unique identifier used for C function and task naming.
  - `type`: Categorizes the task as HARD (HRT) or SOFT (SRT).

- `start_tick` & `end_tick`: Defines the precise execution window relative to the sub-frame.
- `stack_size`: Specifies the memory allocation for the task (defaulting to 256 words).

### 4.3 Automated Generator Script (scheduler_generator.py)

The generator script acts as a pre-compiler validation layer. It processes the JSON configuration to ensure that the defined timeline is mathematically and logically sound before producing source code.

#### 4.3.1 Schedulability Analysis

Before code generation, the script performs a rigorous validation check:

1. **Temporal Integrity**: Verifies that for every task, `start_tick < end_tick`, and that the entire window resides within the `[0, TIMELINE_PERIOD]` range.
2. **HRT Isolation**: Guarantees that no two Hard Real-Time tasks have overlapped windows, preserving the strict isolation required by the `xHardWindowActive` logic.
3. **Utilization Bounds**: Calculates the total CPU budget reserved for HRT tasks. If utilization reaches 100%, the script issues a warning regarding SRT starvation; if it exceeds 100%, generation is aborted to prevent timeline violation.

#### 4.3.2 Artifact Generation (generated_main.c)

Upon successful validation, the script generates the generated_main.c file. This artifact contains:

- **Task Skeletons**: Automated implementation of task functions, including simulated workloads and the mandatory "run-to-completion" self-suspension logic.
- **Configuration Table**: The `TimelineTaskConfig_t` array, fully initialized with the validated timing parameters.
- **System Initialization**: A `main()` function that invokes `vConfigureScheduler()` to register the timeline with the kernel before starting the scheduler.

### 4.4 Development Workflow

The integration of these tools establishes a predictable three-step development cycle:

1. **Definition**: The user modifies system_config.json to reflect the required task timing and constraints.
2. **Validation & Generation**: The generator script is executed: `python3 scheduler_generator.py -c system_config.json.` This step replaces manual, error-prone edits to tasks.c or main.c structures.
3. **Compilation**: The project is built using the provided Makefile, which is pre-configured to compile generated_main.c into the final demo.elf binary.

# 5 CICD and AUTOMATED PIPELINE

## CI/CD and Automated Pipeline

To automate the workflow and quickly validate/visualize the results after each modification, we implemented a CI/CD pipeline. To improve portability, we isolate the runner inside a Docker

container, treated as a lightweight OS environment. This is also useful from a security perspective, since it avoids executing shell commands directly on the development machine. Once the Docker image is built and the container is started, the GitLab Runner must be launched and registered:
- `gitlab-runner start`
-`gitlab-runner register`
During registration, all required fields must be provided, such as the GitLab server URL (in our case `baltig.gitlab`) and the corresponding registration token. After the runner is registered, every push triggers one or more CI jobs that run predefined test instances. The commands executed by the runner are defined in the `.yml` pipeline configuration file. Test validation is performed by checking the program output log and searching for the expected keyword (e.g., `"PASSED"`).

| Status | Job | Stage | Name |
|---|---|---|---|
| passed | #4674 main 08f158ad | test8(Polling) | test8(Polling)_job |
| passed | #4673 main 08f158ad | test7(SemaphoreHa... | test7(SemaphoreHandler)_job |
| passed | #4672 main 08f158ad | test6(DeadlineViola... | test6(DeadlineViolator)_job |
| passed | #4671 main 08f158ad | test5(Multiple_HRT) | test5(Multiple_HRT)_job |
| passed | #4670 main 08f158ad | test4(TimeConsiste... | test4(TimeConsistency)_job |
| passed | #4669 main 08f158ad | test3(Preemption) | test3(Preemption)_job |
| passed | #4668 main 08f158ad | test2(Edge-case) | test2(Edge-case)_job |
| passed | #4667 main 08f158ad | test1(StressTest) | test1(StressTest)_job |
| passed | #4666 main 08f158ad | test0(NormalPriorit... | test0(NormalPriorityBased)_job |
| passed | #4665 main 08f158ad | start | start_job |